PRN232 – Final Assignment · FPT University · FA26

EXE Team Matching System Thiết kế Database

PostgreSQL 16 · 1 instance · mỗi service 1 database riêng

Nguồn: `claude/PROJECT_CONTEXT.md`, `claude/C4_DIAGRAMS.md` (sơ đồ C1) · Phiên bản 19/10/2026

1. Tổng quan & quyền sở hữu

Theo C1, mỗi microservice sở hữu dữ liệu của mình và **không service nào đọc dữ liệu của service khác**. Mỗi "DB" trong C1 là **một database riêng** trên cùng một instance PostgreSQL (container `postgres` trong Docker Compose), nên khi demo có thể mở từng database để xem. Mỗi service kết nối bằng một login role chỉ có quyền CONNECT vào database của mình; không có khóa ngoại chéo database.

Trong mỗi database, bảng nằm trong schema cùng tên service (ví dụ `identity_db` → schema `identity`), nên tên bảng trong tài liệu giữ dạng `schema.bảng`.

Service	"DB" trong C1	Database	Schema	Login role	Bảng
Identity Service	Identity DB	`identity_db`	`identity`	`identity_svc`	roles, user_accounts, refresh_tokens, processed_events
Academic Service	Academic DB	`academic_db`	`academic`	`academic_svc`	semesters, registration_rounds, students, semester_enrollments, import_batches, lecturers, specializations, domains, class_codes, outbox_messages
Team Service	Team DB	`team_db`	`team`	`team_svc`	teams, team_members, team_domains, invitations, join_requests, recruitment_posts (+ specializations), seeking_posts (+ domains), change_requests, change_request_items, lecturer_preferences, lecturer_assignments, audit_logs, outbox_messages
Notification Service	Notification DB	`notification_db`	`notification`	`notification_svc`	contacts, notifications, email_logs, processed_events
Job Service (Hangfire)	Job DB	`job_db`	`hangfire`	`job_svc`	Bảng do Hangfire.PostgreSql tự tạo (job, state, set, hash, lock...). Không có dữ liệu nghiệp vụ.
Matching Service	— (chỉ Redis)	—	—	—	Không có SQL. Read model trong Redis (mục 9).
API Gateway	—	—	—	—	Không lưu dữ liệu.

2. Quy ước chung

- **Đặt tên:** snake_case, bảng số nhiều; `pk_/fk_/uq_/ck_/ex_/idx_` cho ràng buộc và index. Tên ràng buộc ổn định để service map lỗi PostgreSQL sang ProblemDetails.
- **Khóa chính:** `uuid` (`gen_random_uuid()`) cho dữ liệu nghiệp vụ, vì ID đi qua Kafka và giữa các service; `integer IDENTITY` cho danh mục nhỏ; `bigint IDENTITY` cho audit log.
- **Thời gian:** luôn `timestamptz`, lưu UTC. Bảng có thay đổi có `created_at`, `updated_at` (trigger tự cập nhật).
- **Trạng thái/enum:** `varchar` + `CHECK (... IN (...))`, không dùng kiểu ENUM của PostgreSQL.
- **Email:** `citext` để UNIQUE không phân biệt hoa thường. **SĐT:** `varchar(15)`, giữ số 0 đầu.
- **Tham chiếu chéo service:** chỉ lưu ID (không FK) + **snapshot** vài trường cần hiển thị/kiểm tra (MSSV, tên, chuyên ngành, mã domain).
- **Trạng thái hữu hạn:** dùng cột `status` thay cho xóa mềm; dữ liệu có ý nghĩa lịch sử không bị xóa cứng.
- **Ràng buộc 2 lớp:** Services (FluentValidation + Matching `ValidateComposition`) kiểm tra trước để trả lỗi thân thiện; CHECK / UNIQUE / trigger trong DB là lưới an toàn cuối, chống race condition khi nhiều request song song.
- **Optimistic concurrency:** dùng cột hệ thống `xmin` làm row version cho teams, invitations, join_requests, change_requests.

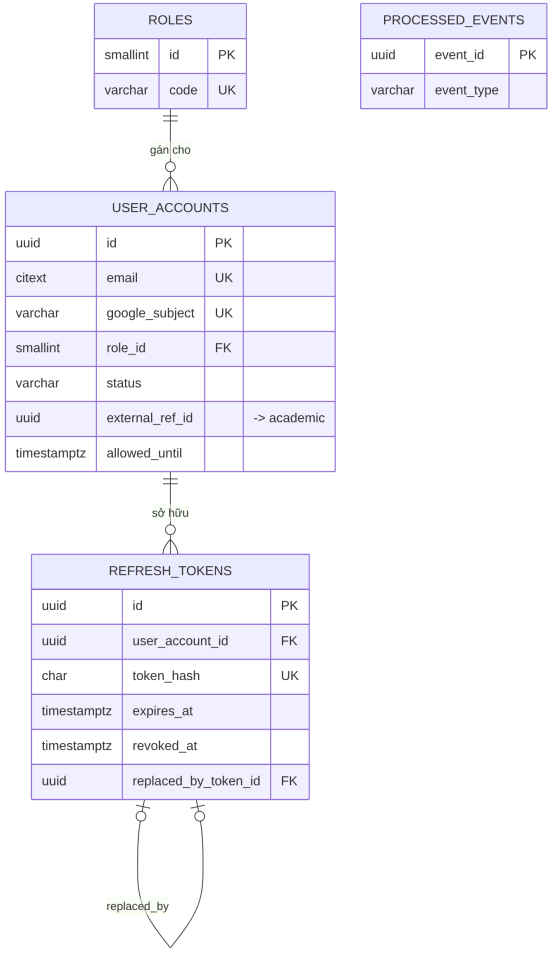
3. Business rules → cách ép trong DB

Rule (PROJECT_CONTEXT §3)	Cách thực thi
1. Nhóm 4–6 người, đúng 1 leader là thành viên	`team_members.role` = LEADER/MEMBER; partial unique `uq_team_members_one_leader` (≤ 1 leader); constraint trigger deferred `ck_team_exactly_one_leader` (= 1 leader lúc COMMIT, cho phép đổi leader trong 1 transaction); `teams.member_count` + `CHECK BETWEEN 0 AND 6` chặn người thứ 7 ngay; khi FINALIZED trigger `ck_team_finalized_size` ép 4–6.
2. ≥ 2 chuyên ngành khi chốt	Snapshot `team_members.specialization_id`; trigger `ck_team_finalized_specializations` đếm DISTINCT khi status = FINALIZED (kể cả khi thay thành viên sau đó). Hard block.
3. Không cần cùng lớp	Không ràng buộc; lớp chỉ nằm ở `academic.semester_enrollments`.
4. 1 sinh viên tối đa 1 nhóm / học kỳ	`UNIQUE (semester_id, student_id)` trên team_members; FK composite `(team_id, semester_id) → teams(id, semester_id)` đảm bảo semester_id khớp nhóm.
5. Chỉ SV trong danh sách học kỳ được đăng nhập	`academic.semester_enrollments` là nguồn; Identity giữ bản sao whitelist trong `user_accounts` (status, `allowed_until` = ngày kết thúc học kỳ), đồng bộ qua `StudentImported`.
6. Chỉ thao tác khi đợt đăng ký mở	registration_rounds (status, opens_at, deadline_at, EXCLUDE chống chồng đợt). Team hỏi Academic qua HTTP (Redis cache) trước khi ghi; không ép được bằng DB vì nằm ở database khác.
7. Change request EXE201 có lý do, Admin duyệt, không giới hạn số người	FK composite `(team_id, course_code)` + `CHECK course_code = 'EXE201'`; `ck_change_requests_reason` (≥ 10 ký tự); `ck_change_requests_review` (APPROVED/REJECTED phải có người và thời điểm duyệt); REJECTED bắt buộc ghi chú; 1 request PENDING / nhóm; số item không giới hạn.
8. Lời mời / join request hết hạn 72h	`expires_at` mặc định `now() + 72h` (service ghi đè theo config); `CHECK expires_at > created_at`; partial index `idx_invitations_expiry` cho job quét PENDING quá hạn.
9. Nguyện vọng GV chỉ tham khảo, Admin quyết	Tách `lecturer_preferences` (≤ 3, ưu tiên 1–3) và `lecturer_assignments` (Admin gán, GV xác nhận; ≤ 1 phân công hiệu lực).
10. Audit log mọi thay đổi nhóm	`team.audit_logs` ghi cùng transaction với thay đổi; trigger chặn UPDATE/DELETE (append-only).
11. MSSV 2 chữ + 6 số; SĐT dạng chuỗi	`CHECK student_code ~ '^[A-Z]{2}[0-9]{6}$'` (academic.students, team snapshot); `phone varchar(15)` + `CHECK ~ '^\+?[0-9]{9,14}$'`. Quyền xem SĐT xử lý ở tầng API.
12. Chuyên ngành / domain chọn từ danh mục	Trong academic: FK tới `specializations`, `domains` (mã D01–D18 có CHECK). Trong team: lưu ID danh mục, Team kiểm tra với Academic khi ghi; `domain_code` có CHECK định dạng.

4. Schema `identity` — Identity Service

Database: `identity_db`, schema `identity`. Owner role: `identity_svc` (chỉ kết nối được `identity_db`).

ERD identity (rút gọn cột; đầy đủ ở bảng bên dưới)



identity.roles

Danh mục vai trò: Student, Lecturer, Admin. Leader không phải role mà là policy ở tầng Services.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	smallint		PK		
code	varchar(20)		UQ		
name	varchar(50)				

Ràng buộc

- CHECK **ck_roles_code** CHECK ((code IN ('Student', 'Lecturer', 'Admin')))
- UQ **uq_roles_code** UNIQUE (code)

identity.user_accounts

Tài khoản đăng nhập kiêm email whitelist (rule 5). Student/Lecturer được tạo sẵn (PENDING) từ event của Academic, chuyển ACTIVE ở lần đăng nhập Google đầu tiên.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
email	citext		UQ		
google_subject	varchar(255)	✓	UQ		"sub" của Google ID token
full_name	varchar(150)				
role_id	smallint		FK		
status	varchar(20)			PENDING	
external_ref_id	uuid	✓			students.id / lecturers.id (Academic), đưa vào JWT claim
allowed_until	timestamptz	✓			Hết hạn whitelist = end_date học kỳ
last_login_at	timestamptz	✓			
created_at	timestamptz			now()	
updated_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_user_accounts_email** CHECK ((email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$'))
- CHECK **ck_user_accounts_external_ref** CHECK ((((role_id IN (1, 2) AND (external_ref_id IS NOT NULL)) OR ((role_id = 3) AND (external_ref_id IS NULL))))
- CHECK **ck_user_accounts_status** CHECK ((status IN ('PENDING', 'ACTIVE', 'LOCKED')))
- FK **fk_user_accounts_role** FOREIGN KEY (role_id) REFERENCES identity.roles(id) ON DELETE RESTRICT
- UQ **uq_user_accounts_email** UNIQUE (email)
- UQ **uq_user_accounts_google_subject** UNIQUE (google_subject)

Index

- **idx_user_accounts_role_id** (role_id)
- **uq_user_accounts_external_ref** UNIQUE (external_ref_id) WHERE (external_ref_id IS NOT NULL)

identity.refresh_tokens

Refresh token ~7 ngày, chỉ lưu SHA-256. Rotation: token cũ bị revoke và trỏ sang token mới.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
user_account_id	uuid		FK		
token_hash	char(64)		UQ		SHA-256 hex
expires_at	timestamptz				

revoked_at	timestamptz	✓			
replaced_by_token_id	uuid	✓	FK		
created_by_ip	inet	✓			
created_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_refresh_tokens_expiry** CHECK ((expires_at > created_at))
- FK **fk_refresh_tokens_replaced_by** FOREIGN KEY (replaced_by_token_id) REFERENCES identity.refresh_tokens(id) ON DELETE SET NULL
- FK **fk_refresh_tokens_user_account** FOREIGN KEY (user_account_id) REFERENCES identity.user_accounts(id) ON DELETE CASCADE
- UQ **uq_refresh_tokens_token_hash** UNIQUE (token_hash)

Index

- **idx_refresh_tokens_replaced_by** (replaced_by_token_id)
- **idx_refresh_tokens_user_account_id** (user_account_id) WHERE (revoked_at IS NULL)

identity.processed_events

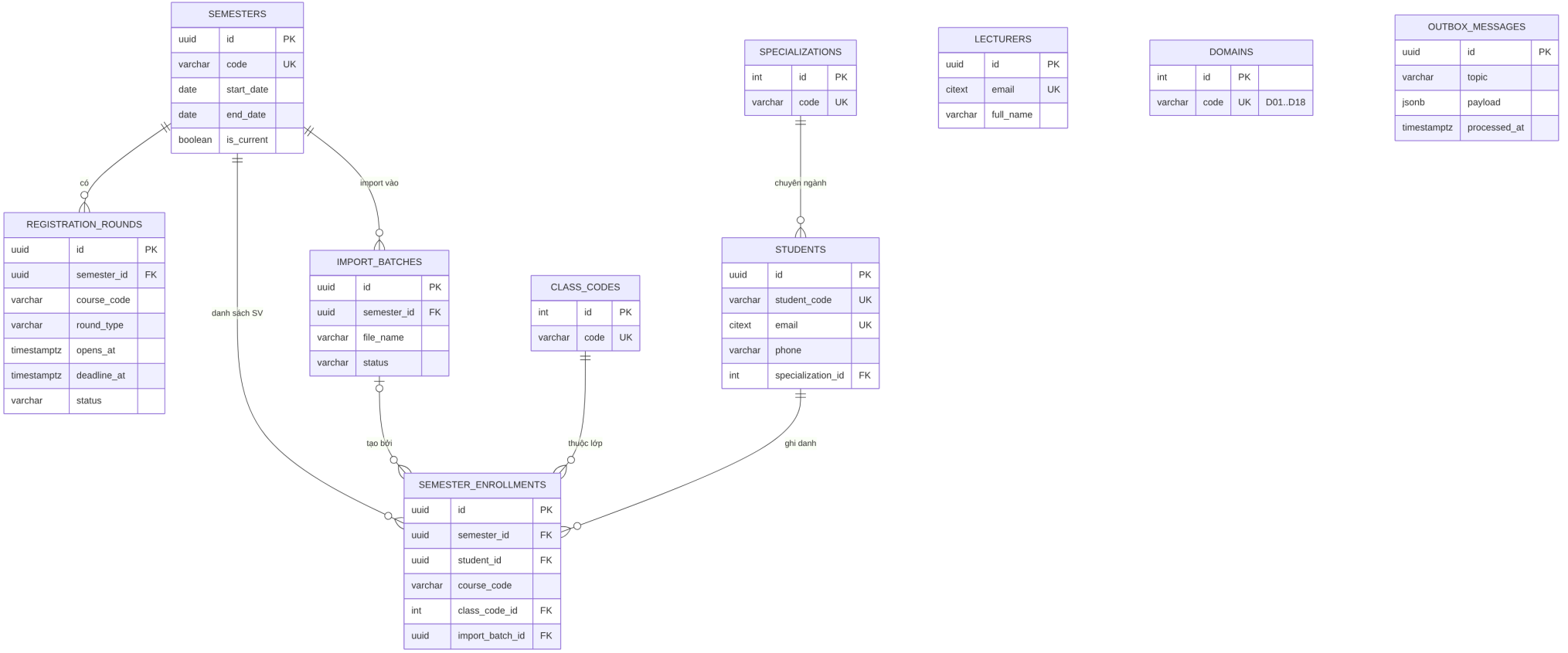
Idempotency khi consume StudentImported / LecturerRegistered.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
event_id	uuid		PK		
event_type	varchar(100)				
processed_at	timestamptz			now()	

5. Schema `academic` — Academic Service

Database: `academic_db`, schema `academic`. Owner role: `academic_svc` (chỉ kết nối được `academic_db`).

ERD academic (rút gọn cột; đầy đủ ở bảng bên dưới)



academic.semesters

Học kỳ (SP/SU/FA + 2 số). Tối đa 1 học kỳ hiện hành.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
code	varchar(4)		UQ		
name	varchar(50)				
start_date	date				
end_date	date				
is_current	boolean			false	
created_at	timestamptz			now()	
updated_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_semesters_code** CHECK ((code ~ '^(SP|SU|FA)[0-9]{2}$'))
- CHECK **ck_semesters_dates** CHECK ((end_date > start_date))
- UQ **uq_semesters_code** UNIQUE (code)

Index

- **uq_semesters_is_current** UNIQUE (is_current) WHERE is_current

academic.registration_rounds

Đợt đăng ký (rule 6). Các đợt cùng học kỳ + môn + loại không được chồng thời gian (EXCLUDE). Job Service đóng đợt khi tới deadline.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
semester_id	uuid		FK		
course_code	varchar(6)				
round_type	varchar(20)				
name	varchar(100)				
opens_at	timestamptz				
deadline_at	timestamptz				
status	varchar(10)			SCHEDULED	
closed_at	timestamptz	✓			
created_by	uuid				identity.user_accounts.id
created_at	timestamptz			now()	
updated_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_registration_rounds_closed** CHECK (((status = 'CLOSED') = (closed_at IS NOT NULL)))
- CHECK **ck_registration_rounds_course** CHECK ((course_code IN ('EXE101', 'EXE201')))
- CHECK **ck_registration_rounds_course_type** CHECK (((round_type <> 'CHANGE_REQUEST') OR (course_code = 'EXE201')))
- CHECK **ck_registration_rounds_status** CHECK ((status IN ('SCHEDULED', 'OPEN', 'CLOSED')))
- CHECK **ck_registration_rounds_type** CHECK ((round_type IN ('TEAM_FORMATION', 'CHANGE_REQUEST')))
- CHECK **ck_registration_rounds_window** CHECK ((deadline_at > opens_at))
- FK **fk_registration_rounds_semester** FOREIGN KEY (semester_id) REFERENCES academic.semesters(id) ON DELETE RESTRICT
- EXCLUDE **ex_registration_rounds_overlap** EXCLUDE USING gist (semester_id WITH =, course_code WITH =, round_type WITH =, tstzrange(opens_at, deadline_at, '[)') WITH &&)

Index

- **idx_registration_rounds_open** (deadline_at) WHERE (status = 'OPEN')
- **idx_registration_rounds_semester_id** (semester_id)

academic.students

Hồ sơ sinh viên, ổn định qua các học kỳ (EXE101 → EXE201). MSSV 2 chữ + 6 số, SĐT là chuỗi.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
student_code	varchar(8)		UQ		MSSV, vd SE184118
full_name	varchar(150)				
email	citext		UQ		
phone	varchar(15)	✓			Chuỗi, giữ số 0 đầu; chỉ đồng đội/GV/Admin xem
specialization_id	integer		FK		
skills	varchar(50)[]			{}	Kỹ năng (tag) cho Matching
bio	varchar(500)	✓			
created_at	timestamptz			now()	
updated_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_students_email** CHECK ((email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$'))
- CHECK **ck_students_phone** CHECK (((phone IS NULL) OR (phone ~ '^\+?[0-9]{9,14}$')))
- CHECK **ck_students_skills_count** CHECK ((cardinality(skills) <= 20))
- CHECK **ck_students_student_code** CHECK ((student_code ~ '^[A-Z]{2}[0-9]{6}$'))
- FK **fk_students_specialization** FOREIGN KEY (specialization_id) REFERENCES academic.specializations(id) ON DELETE RESTRICT
- UQ **uq_students_email** UNIQUE (email)
- UQ **uq_students_student_code** UNIQUE (student_code)

Index

- **idx_students_full_name** (lower(full_name))
- **idx_students_specialization_id** (specialization_id)

academic.semester_enrollments

Danh sách sinh viên của học kỳ = nguồn whitelist. Lớp gắn theo học kỳ.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
semester_id	uuid		FK		
student_id	uuid		FK		
course_code	varchar(6)				
class_code_id	integer		FK		
import_batch_id	uuid	✓	FK		
created_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_semester_enrollments_course** CHECK ((course_code IN ('EXE101', 'EXE201')))
- FK **fk_semester_enrollments_class_code** FOREIGN KEY (class_code_id) REFERENCES academic.class_codes(id) ON DELETE RESTRICT
- FK **fk_semester_enrollments_import_batch** FOREIGN KEY (import_batch_id) REFERENCES academic.import_batches(id) ON DELETE SET NULL

- FK **fk_semester_enrollments_semester** FOREIGN KEY (semester_id) REFERENCES academic.semesters(id) ON DELETE RESTRICT
- FK **fk_semester_enrollments_student** FOREIGN KEY (student_id) REFERENCES academic.students(id) ON DELETE RESTRICT
- UQ **uq_semester_enrollments_semester_student** UNIQUE (semester_id, student_id)

Index

- **idx_semester_enrollments_class_code_id** (class_code_id)
- **idx_semester_enrollments_import_batch** (import_batch_id)
- **idx_semester_enrollments_student_id** (student_id)

academic.import_batches

Mỗi lần import Excel (ClosedXML): số dòng thành công/lỗi, chi tiết lỗi dạng JSON.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
semester_id	uuid		FK		
file_name	varchar(255)				
total_rows	integer			0	
success_rows	integer			0	
failed_rows	integer			0	
errors	jsonb	✓			[{row, field, message}]
status	varchar(20)			PROCESSING	
imported_by	uuid				identity.user_accounts.id
created_at	timestamptz			now()	
completed_at	timestamptz	✓			

Ràng buộc

- CHECK **ck_import_batches_rows** CHECK (((total_rows >= 0) AND (success_rows >= 0) AND (failed_rows >= 0) AND ((success_rows + failed_rows) <= total_rows)))
- CHECK **ck_import_batches_status** CHECK ((status IN ('PROCESSING', 'COMPLETED', 'FAILED')))
- FK **fk_import_batches_semester** FOREIGN KEY (semester_id) REFERENCES academic.semesters(id) ON DELETE RESTRICT

Index

- **idx_import_batches_semester_id** (semester_id)

academic.lecturers

Giảng viên do Admin nhập email (cấp role Lecturer).

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
full_name	varchar(150)				
email	citext		UQ		
phone	varchar(15)	✓			
is_active	boolean			true	
created_at	timestamptz			now()	
updated_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_lecturers_email** CHECK ((email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$'))
- CHECK **ck_lecturers_phone** CHECK (((phone IS NULL) OR (phone ~ '^\+?[0-9]{9,14}$')))
- UQ **uq_lecturers_email** UNIQUE (email)

academic.specializations

Danh mục chuyên ngành hẹp (rule 12).

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	integer		PK	IDENTITY	
code	varchar(20)		UQ		
name	varchar(100)				
is_active	boolean			true	
created_at	timestamptz			now()	
updated_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_specializations_code** CHECK ((code ~ '^[A-Z0-9_]{2,20}$'))
- UQ **uq_specializations_code** UNIQUE (code)

academic.domains

Danh mục domain D01–D18 (rule 12).

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	integer		PK	IDENTITY	
code	varchar(3)		UQ		
name	varchar(150)				
description	varchar(500)	✓			
is_active	boolean			true	
created_at	timestamptz			now()	
updated_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_domains_code** CHECK ((code ~ '^D(0[1-9]|1[0-8])$'))
- UQ **uq_domains_code** UNIQUE (code)

academic.class_codes

Danh mục mã lớp.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	integer		PK	IDENTITY	
code	varchar(20)		UQ		
is_active	boolean			true	
created_at	timestamptz			now()	
updated_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_class_codes_code** CHECK ((code ~ '^[A-Z0-9]{3,20}$'))
- UQ **uq_class_codes_code** UNIQUE (code)

academic.outbox_messages

Transactional Outbox cho StudentImported, StudentProfileUpdated, LecturerRegistered, RoundOpened, RoundClosed.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK		= eventId của envelope

topic	varchar(100)				
event_type	varchar(100)				
aggregate_type	varchar(50)				
aggregate_id	uuid				Kafka message key
payload	jsonb				
correlation_id	varchar(64)	✓			
occurred_at	timestamptz			now()	
processed_at	timestamptz	✓			
attempts	integer			0	
last_error	text	✓			

Ràng buộc

- CHECK **ck_outbox_messages_attempts** CHECK ((attempts >= 0))

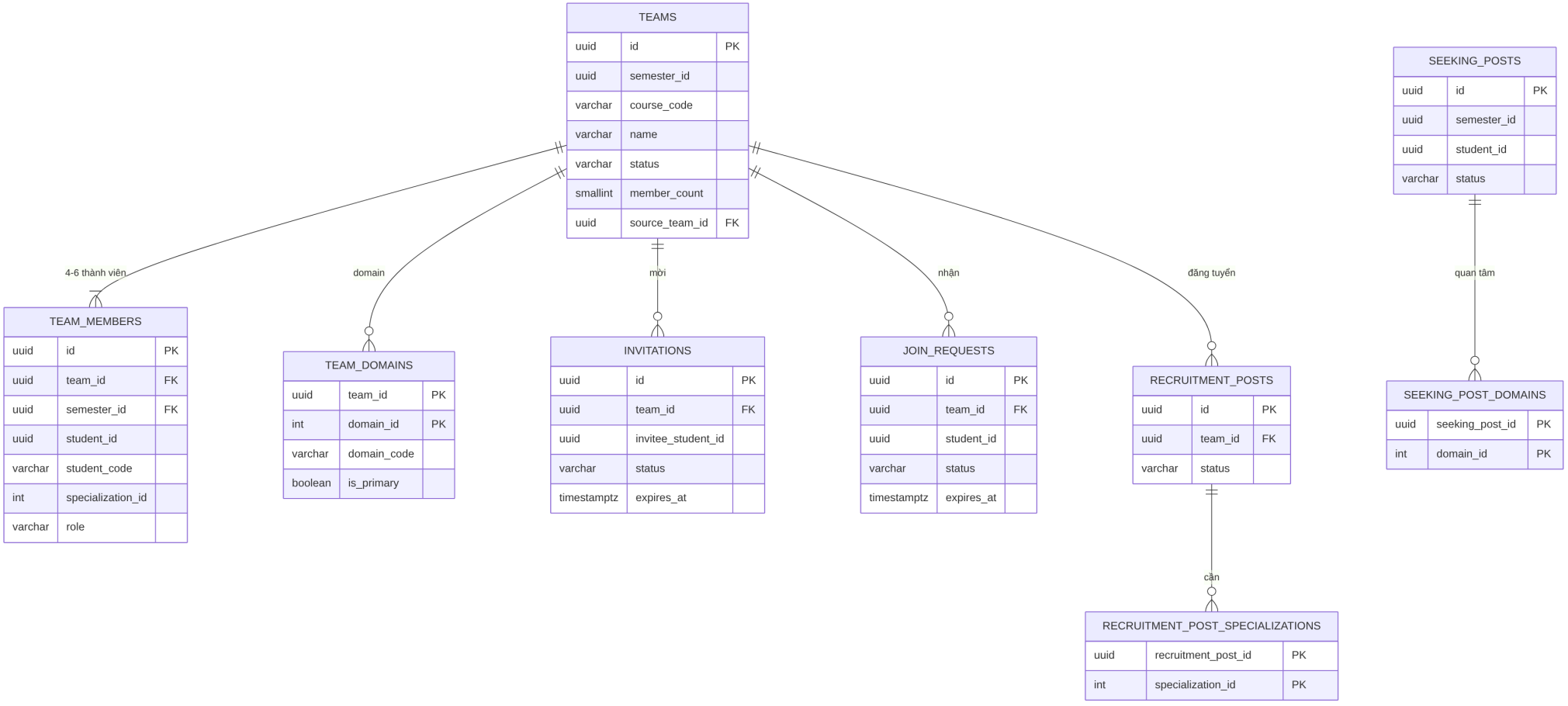
Index

- **idx_outbox_messages_pending** (occurred_at) WHERE (processed_at IS NULL)

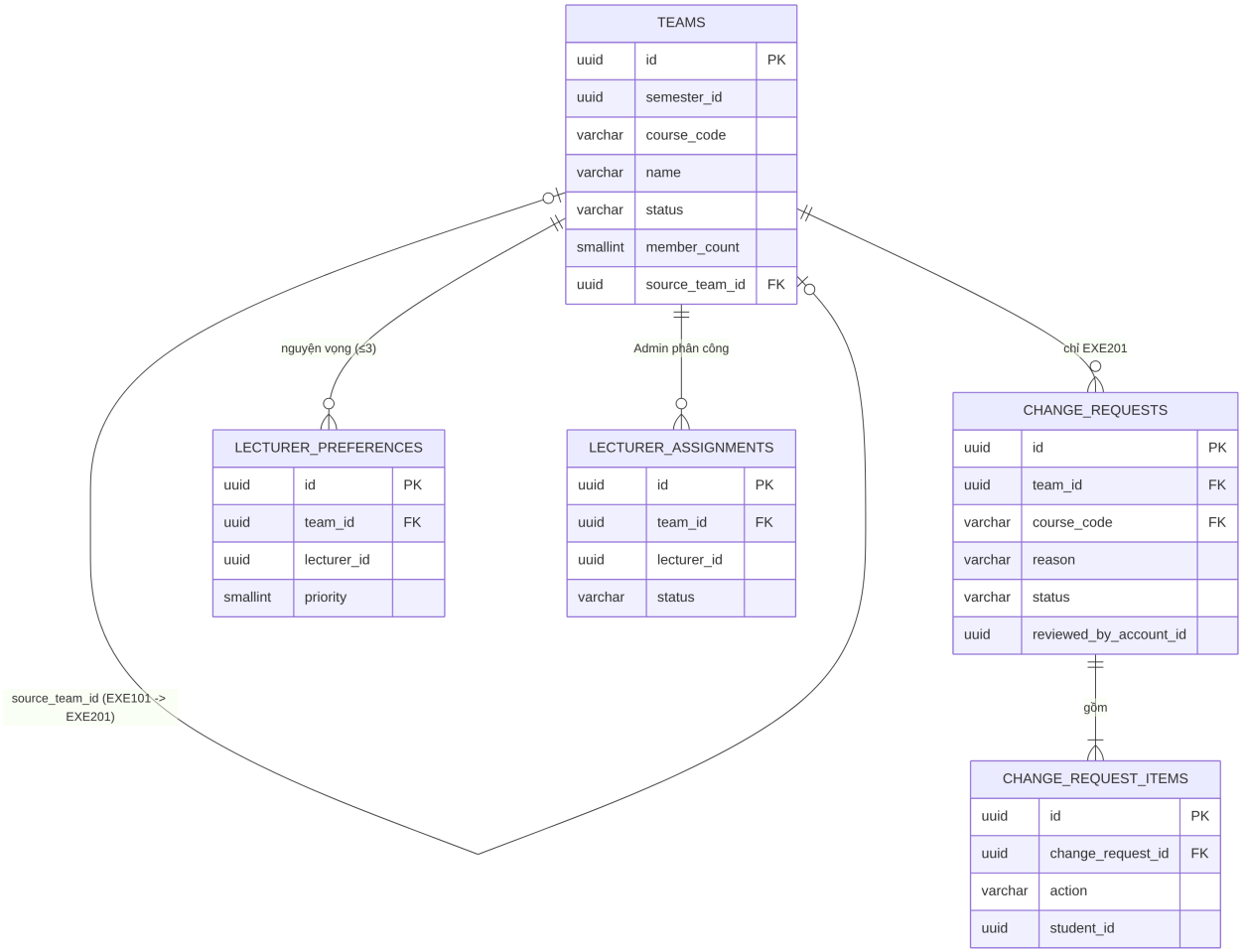
6. Schema `team` — Team Service

Database: `team_db`, schema `team`. Owner role: `team_svc` (chỉ kết nối được `team_db`).

ERD team_core (rút gọn cột; đầy đủ ở bảng bên dưới)



ERD team_change (rút gọn cột; đầy đủ ở bảng bên dưới)



team.teams

Nhóm. member_count do trigger duy trì; CHECK ≤ 6 chặn ngay. Khi FINALIZED: 4–6 người, ≥ 2 chuyên ngành (constraint trigger lúc COMMIT).

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
semester_id	uuid				academic.semesters.id
course_code	varchar(6)				
name	varchar(100)				
idea_title	varchar(200)	✓			
idea_description	varchar(4000)	✓			
status	varchar(12)			FORMING	
member_count	smallint			0	Trigger duy trì
source_team_id	uuid	✓	FK		Nhóm EXE101 gốc
is_auto_assigned	boolean			false	
finalized_at	timestamptz	✓			
created_at	timestamptz			now()	
updated_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_teams_course** CHECK ((course_code IN ('EXE101', 'EXE201')))
- CHECK **ck_teams_finalized_at** CHECK (((status <> 'FINALIZED') OR (finalized_at IS NOT NULL)))
- CHECK **ck_teams_member_count** CHECK (((member_count >= 0) AND (member_count <= 6)))
- CHECK **ck_teams_name** CHECK ((char_length(btrim(name)) >= 2))
- CHECK **ck_teams_source** CHECK (((source_team_id IS NULL) OR (course_code = 'EXE201')))
- CHECK **ck_teams_status** CHECK ((status IN ('FORMING', 'FINALIZED', 'DISBANDED')))
- FK **fk_teams_source_team** FOREIGN KEY (source_team_id) REFERENCES team.teams(id) ON DELETE RESTRICT
- UQ **uq_teams_id_course** UNIQUE (id, course_code)
- UQ **uq_teams_id_semester** UNIQUE (id, semester_id)

Index

- **idx_teams_semester_member_count** (semester_id, member_count) WHERE (status = 'FORMING')
- **idx_teams_semester_status_name** (semester_id, status, name)
- **uq_teams_semester_name** UNIQUE (semester_id, lower(name)) WHERE (status <> 'DISBANDED')
- **uq_teams_source_team_id** UNIQUE (source_team_id) WHERE (source_team_id IS NOT NULL)

Trigger

trg_teams_composition

team.team_members

Thành viên hiện tại. UNIQUE(semester_id, student_id) = rule 4. Partial unique 1 LEADER/nhóm; trigger ép đúng 1 leader. Snapshot MSSV, tên, chuyên ngành.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
team_id	uuid		FK		
semester_id	uuid		FK		Denormalize, ép rule 4
student_id	uuid				academic.students.id
student_code	varchar(8)				Snapshot
full_name	varchar(150)				Snapshot
specialization_id	integer				Snapshot, kiểm tra rule 2

role	varchar(10)			MEMBER	
join_source	varchar(20)				
joined_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_team_members_join_source** CHECK ((join_source IN ('CREATOR', 'INVITATION', 'JOIN_REQUEST', 'CHANGE_REQUEST', 'AUTO_ASSIGN', 'INHERITED')))
- CHECK **ck_team_members_role** CHECK ((role IN ('LEADER', 'MEMBER')))
- CHECK **ck_team_members_student_code** CHECK ((student_code ~ '^[A-Z]{2}[0-9]{6}$'))
- FK **fk_team_members_team** FOREIGN KEY (team_id, semester_id) REFERENCES team.teams(id, semester_id) ON DELETE CASCADE
- UQ **uq_team_members_semester_student** UNIQUE (semester_id, student_id)

Index

- **idx_team_members_team_id** (team_id)
- **uq_team_members_one_leader** UNIQUE (team_id) WHERE (role = 'LEADER')

Trigger

trg_team_members_composition, trg_team_members_count, trg_team_members_immutable

team.team_domains

Domain của nhóm (từ catalog). domain_code snapshot để filter ?domain=D04.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
team_id	uuid		PK FK		
domain_id	integer		PK		academic.domains.id
domain_code	varchar(3)				Snapshot
is_primary	boolean			false	

Ràng buộc

- CHECK **ck_team_domains_code** CHECK ((domain_code ~ '^D(0[1-9]|1[0-8])$'))
- FK **fk_team_domains_team** FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

Index

- **idx_team_domains_domain_code** (domain_code)
- **uq_team_domains_primary** UNIQUE (team_id) WHERE is_primary

team.invitations

Lời mời của leader. Hết hạn 72h (expires_at, service tính từ config). Chỉ 1 lời mời PENDING / (nhóm, sinh viên).

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
team_id	uuid		FK		
invitee_student_id	uuid				
invited_by_student_id	uuid				Leader
message	varchar(500)	✓			
status	varchar(10)			PENDING	
created_at	timestamptz			now()	
expires_at	timestamptz			(now() + 72:00:00)	created_at + 72h (config)
resolved_at	timestamptz	✓			

Ràng buộc

- CHECK **ck_invitations_expiry** CHECK ((expires_at > created_at))
- CHECK **ck_invitations_resolved** CHECK (((status = 'PENDING') = (resolved_at IS NULL)))

- CHECK **ck_invitations_self** CHECK ((invitee_student_id <> invited_by_student_id))
- CHECK **ck_invitations_status** CHECK ((status IN ('PENDING', 'ACCEPTED', 'DECLINED', 'EXPIRED', 'CANCELLED')))
- FK **fk_invitations_team** FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

Index

- **idx_invitations_expiry** (expires_at) WHERE (status = 'PENDING')
- **idx_invitations_invitee** (invitee_student_id, status)
- **uq_invitations_pending** UNIQUE (team_id, invitee_student_id) WHERE (status = 'PENDING')

team.join_requests

Yêu cầu xin vào nhóm của sinh viên, leader duyệt. Hết hạn 72h.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
team_id	uuid		FK		
student_id	uuid				
message	varchar(500)	✓			
status	varchar(10)			PENDING	
resolved_by_student_id	uuid	✓			Leader duyệt
created_at	timestamptz			now()	
expires_at	timestamptz			(now() + 72:00:00)	
resolved_at	timestamptz	✓			

Ràng buộc

- CHECK **ck_join_requests_expiry** CHECK ((expires_at > created_at))
- CHECK **ck_join_requests_resolved** CHECK (((status = 'PENDING') = (resolved_at IS NULL)))
- CHECK **ck_join_requests_status** CHECK ((status IN ('PENDING', 'ACCEPTED', 'DECLINED', 'EXPIRED', 'CANCELLED')))
- FK **fk_join_requests_team** FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

Index

- **idx_join_requests_expiry** (expires_at) WHERE (status = 'PENDING')
- **idx_join_requests_student** (student_id, status)
- **uq_join_requests_pending** UNIQUE (team_id, student_id) WHERE (status = 'PENDING')

team.recruitment_posts

Bài nhóm tìm thành viên. Tối đa 1 bài OPEN / nhóm.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
team_id	uuid		FK		
title	varchar(150)				
description	varchar(2000)	✓			
status	varchar(10)			OPEN	
created_at	timestamptz			now()	
updated_at	timestamptz			now()	
closed_at	timestamptz	✓			

Ràng buộc

- CHECK **ck_recruitment_posts_closed** CHECK (((status = 'CLOSED') = (closed_at IS NOT NULL)))
- CHECK **ck_recruitment_posts_status** CHECK ((status IN ('OPEN', 'CLOSED')))
- FK **fk_recruitment_posts_team** FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE

Index

- **uq_recruitment_posts_open** UNIQUE (team_id) WHERE (status = 'OPEN')

team.recruitment_post_specializations

Chuyên ngành nhóm đang thiếu.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
recruitment_post_id	uuid		PK FK		
specialization_id	integer		PK		

Ràng buộc

- FK **fk_rps_post** FOREIGN KEY (recruitment_post_id) REFERENCES team.recruitment_posts(id) ON DELETE CASCADE

Index

- **idx_rps_specialization_id** (specialization_id)

team.seeking_posts

Bài sinh viên tìm nhóm. Tối đa 1 bài OPEN / sinh viên / học kỳ.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
semester_id	uuid				
student_id	uuid				
specialization_id	integer				
title	varchar(150)				
description	varchar(2000)	✓			
status	varchar(10)			OPEN	
created_at	timestamptz			now()	
updated_at	timestamptz			now()	
closed_at	timestamptz	✓			

Ràng buộc

- CHECK **ck_seeking_posts_closed** CHECK (((status = 'CLOSED') = (closed_at IS NOT NULL)))
- CHECK **ck_seeking_posts_status** CHECK ((status IN ('OPEN', 'CLOSED')))

Index

- **idx_seeking_posts_semester_spec** (semester_id, specialization_id) WHERE (status = 'OPEN')
- **uq_seeking_posts_open** UNIQUE (semester_id, student_id) WHERE (status = 'OPEN')

team.seeking_post_domains

Domain sinh viên quan tâm.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
seeking_post_id	uuid		PK FK		
domain_id	integer		PK		
domain_code	varchar(3)				

Ràng buộc

- CHECK **ck_spd_code** CHECK ((domain_code ~ '^D(0[1-9]|1[0-8])$'))
- FK **fk_spd_post** FOREIGN KEY (seeking_post_id) REFERENCES team.seeking_posts(id) ON DELETE CASCADE

team.change_requests

Change request EXE201 (rule 7). FK composite (team_id, course_code) + CHECK = 'EXE201' nên nhóm EXE101 không tạo được. Lý do bắt buộc ≥ 10 ký tự. 1 request PENDING / nhóm.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	

team_id	uuid		FK		
course_code	varchar(6)		FK	EXE201	
requested_by_student_id	uuid				
reason	varchar(1000)				
status	varchar(10)			PENDING	
reviewed_by_account_id	uuid	✓			Admin (identity)
reviewed_at	timestamptz	✓			
review_note	varchar(1000)	✓			
created_at	timestamptz			now()	
updated_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_change_requests_course** CHECK ((course_code = 'EXE201'))
- CHECK **ck_change_requests_reason** CHECK ((char_length(btrim(reason)) >= 10))
- CHECK **ck_change_requests_reject_note** CHECK (((status <> 'REJECTED') OR (review_note IS NOT NULL)))
- CHECK **ck_change_requests_review** CHECK (((status IN ('APPROVED', 'REJECTED')) = ((reviewed_by_account_id IS NOT NULL) AND (reviewed_at IS NOT NULL))))
- CHECK **ck_change_requests_status** CHECK ((status IN ('PENDING', 'APPROVED', 'REJECTED', 'CANCELLED')))
- FK **fk_change_requests_team** FOREIGN KEY (team_id, course_code) REFERENCES team.teams(id, course_code) ON DELETE RESTRICT

Index

- **idx_change_requests_status_created** (status, created_at DESC)
- **uq_change_requests_pending** UNIQUE (team_id) WHERE (status = 'PENDING')

team.change_request_items

Từng thay đổi: ADD / REMOVE / SET_LEADER. Không giới hạn số lượng.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
change_request_id	uuid		FK		
action	varchar(12)				
student_id	uuid				
student_code	varchar(8)				
full_name	varchar(150)				
specialization_id	integer	✓			Bắt buộc với ADD

Ràng buộc

- CHECK **ck_change_request_items_action** CHECK ((action IN ('ADD', 'REMOVE', 'SET_LEADER')))
- CHECK **ck_change_request_items_code** CHECK ((student_code ~ '^[A-Z]{2}[0-9]{6}$'))
- CHECK **ck_change_request_items_spec** CHECK (((action <> 'ADD') OR (specialization_id IS NOT NULL)))
- FK **fk_change_request_items_request** FOREIGN KEY (change_request_id) REFERENCES team.change_requests(id) ON DELETE CASCADE
- UQ **uq_change_request_items** UNIQUE (change_request_id, student_id, action)

Index

- **idx_change_request_items_student_id** (student_id)

team.lecturer_preferences

Nguyện vọng GV của nhóm, tối đa 3, chỉ tham khảo (rule 9).

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	

team_id	uuid		FK		
lecturer_id	uuid				academic.lecturers.id
priority	smallint				
created_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_lecturer_preferences_priority** CHECK (((priority >= 1) AND (priority <= 3)))
- FK **fk_lecturer_preferences_team** FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE CASCADE
- UQ **uq_lecturer_preferences_team_lecturer** UNIQUE (team_id, lecturer_id)
- UQ **uq_lecturer_preferences_team_priority** UNIQUE (team_id, priority)

Index

- **idx_lecturer_preferences_lecturer_id** (lecturer_id)

team.lecturer_assignments

Admin phân công (quyết định cuối), GV xác nhận/từ chối. Tối đa 1 phân công hiệu lực / nhóm.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
team_id	uuid		FK		
lecturer_id	uuid				
lecturer_name	varchar(150)				Snapshot
status	varchar(10)			ASSIGNED	
assigned_by_account_id	uuid				Admin (identity)
assigned_at	timestamptz			now()	
responded_at	timestamptz	✓			
note	varchar(500)	✓			

Ràng buộc

- CHECK **ck_lecturer_assignments_responded** CHECK (((status <> ALL ((ARRAY['CONFIRMED', 'DECLINED']))) OR (responded_at IS NOT NULL)))
- CHECK **ck_lecturer_assignments_status** CHECK ((status IN ('ASSIGNED', 'CONFIRMED', 'DECLINED', 'REVOKED')))
- FK **fk_lecturer_assignments_team** FOREIGN KEY (team_id) REFERENCES team.teams(id) ON DELETE RESTRICT

Index

- **idx_lecturer_assignments_lecturer** (lecturer_id, status)
- **uq_lecturer_assignments_active** UNIQUE (team_id) WHERE (status IN ('ASSIGNED', 'CONFIRMED'))

team.audit_logs

Audit log (rule 10): ai, khi nào, thay đổi gì (old/new JSON). Append-only bằng trigger chặn UPDATE/DELETE.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	bigint		PK	IDENTITY	
team_id	uuid	✓			
entity_type	varchar(50)				
entity_id	uuid				
action	varchar(50)				
actor_account_id	uuid	✓			NULL = hệ thống/job
actor_role	varchar(20)				
old_values	jsonb	✓			
new_values	jsonb	✓			
reason	varchar(1000)	✓			

correlation_id	varchar(64)	✓			
occurred_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_audit_logs_actor_role** CHECK ((actor_role IN ('Student', 'Lecturer', 'Admin', 'System')))

Index

- **idx_audit_logs_actor** (actor_account_id)
- **idx_audit_logs_entity** (entity_type, entity_id)
- **idx_audit_logs_team_time** (team_id, occurred_at DESC)

Trigger

trg_audit_logs_append_only

team.outbox_messages

Transactional Outbox: ghi cùng transaction với thay đổi nghiệp vụ; dispatcher nền đẩy sang Kafka.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK		= eventId của envelope
topic	varchar(100)				
event_type	varchar(100)				
aggregate_type	varchar(50)				
aggregate_id	uuid				Kafka message key
payload	jsonb				
correlation_id	varchar(64)	✓			
occurred_at	timestamptz			now()	
processed_at	timestamptz	✓			
attempts	integer			0	
last_error	text	✓			

Ràng buộc

- CHECK **ck_outbox_messages_attempts** CHECK ((attempts >= 0))

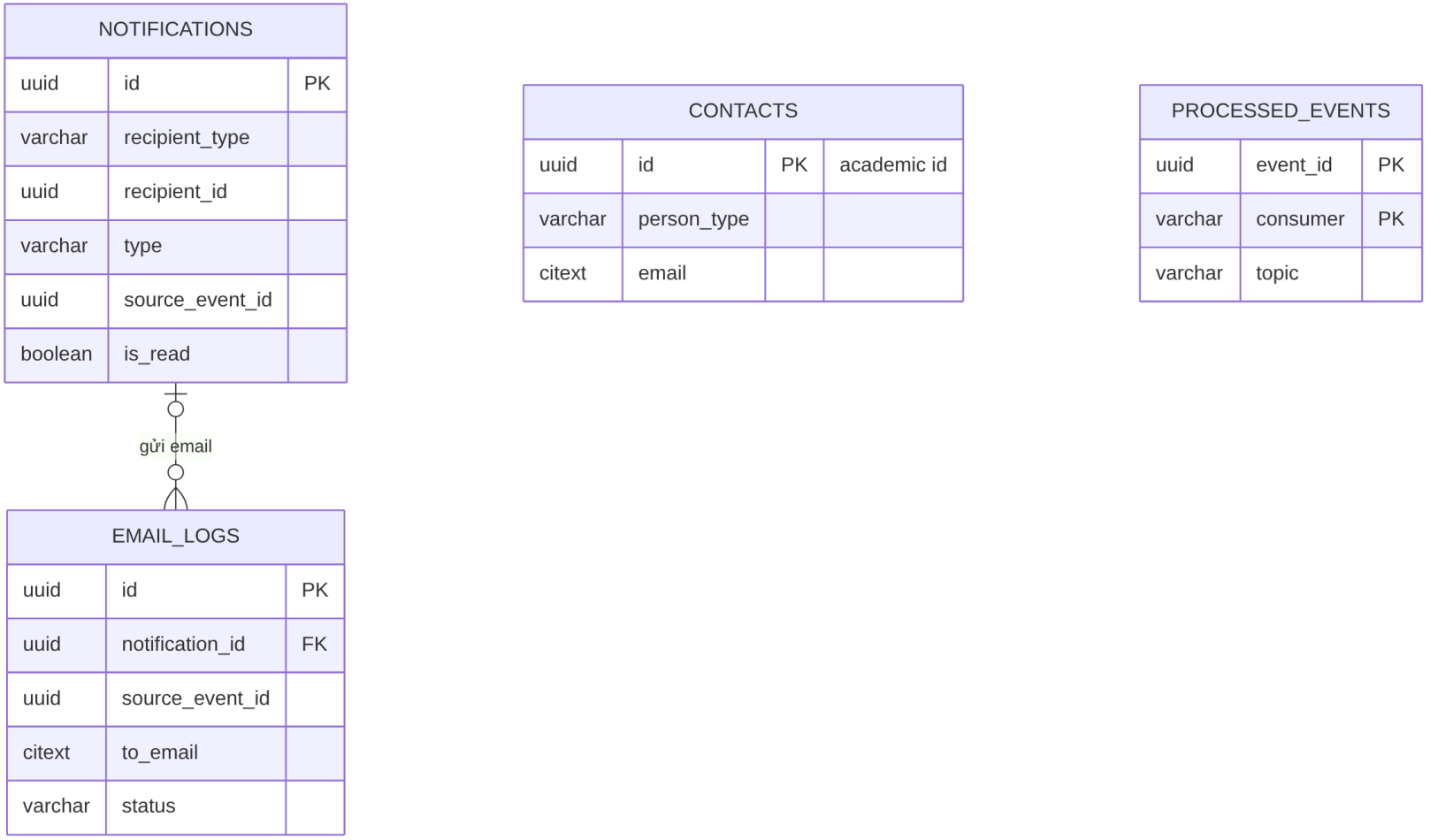
Index

- **idx_outbox_messages_pending** (occurred_at) WHERE (processed_at IS NULL)

7. Schema `notification` — Notification Service

Database: `notification_db`, schema `notification`. Owner role: `notification_svc` (chỉ kết nối được `notification_db`).

ERD notification (rút gọn cột; đầy đủ ở bảng bên dưới)



notification.contacts

Read model người nhận (email, tên) dựng từ topic exe.student.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK		academic students.id / lecturers.id
person_type	varchar(10)				
email	citext				
full_name	varchar(150)				
updated_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_contacts_person_type** CHECK ((person_type IN ('STUDENT', 'LECTURER')))

Index

- **idx_contacts_email** (email)

notification.notifications

Thông báo in-app. ADMIN → recipient_id NULL (thông báo chung). UNIQUE (event, người nhận) để idempotent.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
recipient_type	varchar(10)				
recipient_id	uuid	✓			academic id; NULL nếu ADMIN
type	varchar(50)				= eventType
title	varchar(200)				
content	varchar(2000)				
link_url	varchar(500)	✓			
source_event_id	uuid				
is_read	boolean			false	
read_at	timestamptz	✓			
created_at	timestamptz			now()	

Ràng buộc

- CHECK **ck_notifications_read** CHECK ((is_read = (read_at IS NOT NULL)))
- CHECK **ck_notifications_recipient** CHECK (((recipient_type = 'ADMIN') = (recipient_id IS NULL)))
- CHECK **ck_notifications_recipient_type** CHECK ((recipient_type IN ('STUDENT', 'LECTURER', 'ADMIN')))

Index

- **idx_notifications_recipient** (recipient_type, recipient_id, created_at DESC)
- **idx_notifications_unread** (recipient_type, recipient_id) WHERE (NOT is_read)
- **uq_notifications_event_recipient** UNIQUE (source_event_id, recipient_type, recipient_id) NULLS NOT DISTINCT

notification.email_logs

Nhật ký gửi email SMTP; retry khi FAILED. UNIQUE (event, email).

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
id	uuid		PK	gen_random_uuid()	
notification_id	uuid	✓	FK		
source_event_id	uuid				
to_email	citext				

subject	varchar(255)				
template_code	varchar(50)				
status	varchar(10)			PENDING	
attempts	integer			0	
last_error	text	✓			
created_at	timestamptz			now()	
sent_at	timestamptz	✓			

Ràng buộc

- CHECK **ck_email_logs_attempts** CHECK ((attempts >= 0))
- CHECK **ck_email_logs_sent** CHECK (((status = 'SENT') = (sent_at IS NOT NULL)))
- CHECK **ck_email_logs_status** CHECK ((status IN ('PENDING', 'SENT', 'FAILED')))
- FK **fk_email_logs_notification** FOREIGN KEY (notification_id) REFERENCES notification.notifications(id) ON DELETE SET NULL

Index

- **idx_email_logs_notification_id** (notification_id)
- **idx_email_logs_retry** (created_at) WHERE (status IN ('PENDING', 'FAILED'))
- **uq_email_logs_event_to** UNIQUE (source_event_id, to_email)

notification.processed_events

EventId đã xử lý theo từng handler, chống gửi trùng.

Cột	Kiểu	Null	Khóa	Mặc định	Ghi chú
event_id	uuid		PK		
consumer	varchar(100)		PK		Tên handler
topic	varchar(100)				
event_type	varchar(100)				
processed_at	timestamptz			now()	

Index

- **idx_processed_events_processed_at** (processed_at)

8. Tham chiếu chéo service & đồng bộ qua Kafka

8.1 ID tham chiếu sang service khác

Cột (schema chứa)	Trỏ tới (chủ sở hữu)	Kiểm tra khi ghi	Đồng bộ
`team.*.student_id, invitee_student_id, requested_by_student_id...`	academic.students.id	Team gọi Academic HTTP "SV có trong học kỳ?"; lấy snapshot MSSV, tên, chuyên ngành	Snapshot lúc ghi. Khi chốt nhóm, Team lấy lại chuyên ngành từ Academic
`team.teams.semester_id, seeking_posts.semester_id`	academic.semesters.id	HTTP "đợt đăng ký có đang mở?"	Không cần
`team.*.specialization_id, domain_id (+ domain_code)`	academic.specializations / domains	HTTP lấy danh mục (Academic cache Redis)	Danh mục ít đổi, chỉ vô hiệu (is_active) chứ không xóa
`team.lecturer_preferences / lecturer_assignments.lecturer_id`	academic.lecturers.id	HTTP kiểm tra GV đang hoạt động	Snapshot lecturer_name
`*_account_id, created_by, imported_by`	identity.user_accounts.id	Lấy từ claim `sub` trong JWT	Không cần
`identity.user_accounts.external_ref_id`	academic.students.id / lecturers.id	—	Consume `StudentImported`, `LecturerRegistered`
`notification.contacts.id, notifications.recipient_id`	academic.students.id / lecturers.id	—	Consume topic `exe.student`

JWT do Identity cấp mang các claim: `sub` (user_account id), `role`, `student_id` hoặc `lecturer_id` (= external_ref_id). Nhờ vậy Team và Notification làm việc trực tiếp bằng ID của Academic.

8.2 Event, producer và consumer

Topic	Event	Producer (cách ghi)	Consumer → dữ liệu cập nhật
`exe.student`	StudentImported, StudentProfileUpdated, **LecturerRegistered***	Academic (outbox)	Identity → user_accounts; Notification → contacts; Matching → Redis
`exe.registration`	RoundOpened, RoundClosed	Academic (outbox)	Notification → notifications/email; Job → lên lịch RoundAutoCloseJob, AutoAssignUnmatchedJob*
`exe.registration`	DeadlineApproaching	Job (publish trực tiếp, không có dữ liệu nghiệp vụ)	Notification

exe.team	TeamCreated, MemberJoined, MemberLeft, LeaderChanged, TeamFinalized	Team (outbox)	Matching → Redis; Notification
exe.invitation	InvitationSent/Accepted/Declined/Expired, JoinRequestSubmitted	Team (outbox)*	Notification
exe.change-request	ChangeRequestSubmitted/Approved/Rejected	Team (outbox)	Notification
exe.assignment	LecturerRequested, LecturerAccepted, LecturerAssigned	Team (outbox)	Notification

* Khác/bổ sung so với tài liệu hiện có — xem mục 10.

8.3 Transactional Outbox

- Service có dữ liệu nghiệp vụ và publish event (**Team, Academic**) có bảng `outbox_messages` trong schema của mình. Thay đổi nghiệp vụ + audit log + dòng outbox ghi trong **cùng một transaction**.
- `outbox_messages.id` chính là `eventId` của envelope; `aggregate_id` (team_id, student_id, round_id) làm Kafka key để giữ thứ tự theo đối tượng.
- Dispatcher nền mỗi 1–2 giây: `SELECT ... WHERE processed_at IS NULL ORDER BY occurred_at LIMIT 100 FOR UPDATE SKIP LOCKED` → publish → set `processed_at`; lỗi thì tăng `attempts`, ghi `last_error`. Dọn dòng đã xử lý sau 7 ngày.
- Consumer idempotent: Identity và Notification lưu eventId trong `processed_events` cùng transaction với thay đổi; Notification còn có UNIQUE (event, người nhận) trên notifications và email_logs. Lỗi sau khi retry → `<topic>.dlq`.
- Job Service không có outbox: event của nó (DeadlineApproaching) không gắn với thay đổi dữ liệu nào, publish trực tiếp là đủ.

9. Service không có schema nghiệp vụ

9.1 Matching Service — Redis read model

Key	Kiểu	Nội dung	Cập nhật bởi event
`matching:{semesterId}:student:{studentId}`	Hash	specializationId, skills, teamId (rỗng = chưa có nhóm)	StudentImported, StudentProfileUpdated, MemberJoined, MemberLeft
`matching:{semesterId}:unmatched:{specializationId}`	Set	studentId chưa có nhóm theo chuyên ngành	StudentImported, MemberJoined, MemberLeft
`matching:{semesterId}:team:{teamId}`	Hash	memberCount, specializations (đếm theo chuyên ngành), status	TeamCreated, MemberJoined, MemberLeft, TeamFinalized
`matching:processed:{eventId}`	String (TTL 7 ngày)	Idempotency	Mọi event

Không TTL cho read model; có thể dựng lại bằng cách replay topic `exe.student` và `exe.team` từ đầu.

9.2 Job Service — schema hangfire

Cấu hình `UsePostgreSqlStorage(..., new PostgreSqlStorageOptions { SchemaName = "hangfire", PrepareSchemaIfNecessary = true })` với role `job_svc`. Hangfire tự tạo và quản lý bảng; không thêm bảng nghiệp vụ.

10. Giả định (chưa có trong tài liệu)

1. **Mỗi service 1 database riêng** trên cùng một instance PostgreSQL (identity_db, academic_db, team_db, notification_db, job_db), đúng PROJECT_CONTEXT §9; chọn cách này để demo mở từng database cho dễ. Cô lập bằng role + quyền CONNECT; extension citext, btree_gist cài trong từng database cần dùng.
2. **Email whitelist (đã chốt phương án b):** Academic publish StudentImported, Identity consume để tạo sẵn tài khoản; C1 đã có cạnh Kafka → Identity. Login hợp lệ khi status ≠ LOCKED và (role ≠ Student hoặc allowed_until > now()).
3. **Event mới `LecturerRegistered`** (topic exe.student) khi Admin thêm email giảng viên, để Identity cấp role Lecturer.
4. **Job không đụng DB nghiệp vụ:** InvitationExpiryJob gọi endpoint nội bộ của Team (service token) để đánh dấu hết hạn, Team publish `InvitationExpired` qua outbox (tài liệu ghi Job là producer). Tương tự RoundAutoCloseJob gọi Academic để đóng đợt, Academic publish `RoundClosed`. Job consume `Round0pened` để lên lịch. Các cạnh Job → Team/Academic và Kafka → Job chưa có trong C1.
5. **Notification consume exe.student** để có email người nhận (bảng contacts); tài liệu chỉ ghi Matching consume topic này.
6. **Snapshot không tự cập nhật trong Team** (Team không consume Kafka theo C1). Khi chốt nhóm, Team lấy lại chuyên ngành mới nhất từ Academic trước khi đổi status.
7. **Nhóm EXE201 được tạo bằng thao tác Admin "kế thừa nhóm"**: sao chép thành viên nhóm EXE101 sang học kỳ mới với `source_team_id`, `join_source = INHERITED`. Sau đó mọi thay đổi thành viên chỉ qua change request được duyệt.
8. **Mỗi sinh viên có đúng 1 chuyên ngành hẹp**; mỗi học kỳ thuộc 1 lớp và 1 môn (EXE101 hoặc EXE201).
9. **Trạng thái nhóm:** FORMING → FINALIZED; DISBANDED khi giải tán (xóa thành viên để họ được vào nhóm khác). Đợt CHANGE_REQUEST chỉ có ở EXE201.
10. **Giới hạn tự đặt:** lý do change request ≥ 10 ký tự, từ chối phải có ghi chú; ≤ 3 nguyện vọng GV; 1 bài tuyển OPEN / nhóm; 1 bài tìm nhóm OPEN / sinh viên / học kỳ; SĐT 9–14 chữ số, có thể có dấu +.
11. **Thông báo cho Admin là thông báo chung** (recipient_id NULL), không theo dõi đã đọc riêng từng Admin.
12. **Refresh token chỉ lưu hash SHA-256**, có rotation.
13. **Dữ liệu seed danh mục** (chuyên ngành, tên domain D01–D18) là tạm, cần thay bằng danh mục thật của phòng EXE. Admin đầu tiên được seed.
14. **Thành viên hiện tại** lưu trong team_members; lịch sử ra/vào nhóm nằm trong audit_logs (không có bảng lịch sử riêng).